



技術レポート

Android向け高忠実度オーディオプレーヤー

Lyra Bitperfectは、音楽を可能な限り高い信号整合性でリスナーへ届けることを目的に、Android向けにゼロから設計されたオーディオプレーヤーです。視覚設計と技術設計のすべてが、妥協のない再生体験に向けられています。物理ノブ、質感のあるディスプレイ、トグルスイッチ、回転するCDといったアナログ機器を思わせるインターフェースに、ファイルとDACの間にある余分な層を排したネイティブ Kotlin オーディオエンジンを組み合わせています。

1. 全体アーキテクチャ

Lyra BitperfectはFlutter/Dartで構築され、Androidのオーディオサブシステムに低レベルでアクセスするためにネイティブ Kotlin レイヤーを備えています。再生エンジンはExoPlayer/Media3を用いて完全にKotlin側で動作し、FlutterはUIレイヤーとしてのみ機能します。両者の通信には専用のネイティブチャンネルを使用します。

1.1 Flutter-Kotlin通信チャンネル

チャンネル	種類	機能
lyra_app/playback	MethodChannel	再生、一時停止、次へ、前へ、シーク、シャッフル、リピート
lyra_app/state_events	EventChannel	再生状態、位置、トラック番号をリアルタイム送信
lyra_app/metadata	MethodChannel	メタデータとアートワークの抽出 (MediaMetadataRetriever)
lyra_app/bitperfect	MethodChannel	Bitperfectモード管理とUSB DAC検出
lyra_app/usb_events	EventChannel	USB接続/切断イベントのリアルタイム配信
lyra_app/rms_events	EventChannel	各メーター用の波形データとRMS配信
lyra_app/volume	MethodChannel	ネイティブ音量制御 - Player と AudioManager
lyra_app/trial	MethodChannel	試用期間と購入フローの管理

1.2 再生エンジン

エンジンはExoPlayer/Media3を用いてKotlinで構築されています。これによりFlutterのオーディオプラグイン層、just_audio、プラグインブリッジ、中間抽象化を使いません。ExoPlayerがプレイリスト、曲間遷移、オーディオセッション、ハードウェア連携を直接管理します。再生/一時停止/前後スキップ時のフェードは、信号そのものを処理することなく、ExoPlayerのボリュームに適用される技術的なゲインランプとして実装されています。

のものを加工せず、ExoPlayerボリューム上の技術的なゲインランプとして実装されています。

2. 再生システムとライブラリ

2.1 ライブラリ管理

ローカルに保存された楽曲を、Songs、Artists、Albums、Folders、Playlistsの5つのビューで管理します。メタデータはインポート時にMediaMetadataRetrieverで取得され、JSONとしてSharedPreferencesに保存されます。

2.2 再生を中断しない設計

ライブラリを閲覧しても再生は中断されません。ユーザーが明示的に曲を選択した場合にのみトラックが切り替わります。何も選ばず戻ると、再生はそのままの位置から継続します。

2.3 自動トラック進行

曲が終了するとExoPlayerがSTATE_ENDEDを発行します。エンジンはこれを検知して次のメディア項目へ進みます。最後の曲が終わった場合は先頭へ戻りますが、自動再生は開始しません。ShuffleとRepeatはExoPlayerがネイティブに処理します。

3. Bitperfectシステム - Lyraの中核

BitperfectシステムはLyraを特徴づける中心機能です。BITPERFECTボタンだけで制御される3つの状態を持ち、それぞれが異なる音響上の意味を持ちます。現在の状態はLEDの色で明確に示されます。

3.1 3つの状態

LED	状態	説明
白/パール	Direct Playback	Androidの標準オーディオチェーン。EQなし、DSPなし、介入なし。
緑	Direct Mode	EQなし。AudioManager経由でhifi_mode=on。内部ハードウェアの透明性を最大化。
青	Bitperfect USB	USB DACを検出。外部DACへビット単位で信号を送出。

3.2 Direct Playback - 白LED

信号処理を行わない標準再生です。Androidのオーディオチェーンは完全に標準のままで、イコライザー、HiFiモード、追加のAudioAttributes最適化は使いません。端末標準のオーディオスタックが出力する信号をそのまま扱います。

3.3 Direct Mode - 緑LED

BITPERFECTを押すと有効になります。Lyraはシステム処理を抑えるようAudioAttributesを設定し、AudioManager.setParameters("hifi_mode=on")でハードウェアHiFiモードを有効にします。目的はデジタル減衰を避け、内部チェーンの透明性を最大化することです。

3.4 Real Bitperfect USB - 青LED

互換USB DACを検出すると自動的に有効になります。信号は内部DACを完全に迂回し、丸め、ミキシング、DSPを行わずに外部ハードウェアへ送られます。USB Audio Class 1/2 (UAC1/UAC2)に対応するデバイスで利用できます。検出はinterfaceClass == 1というハードウェアインターフェース単位で行います。

4. Reference Monitor - HEADROOM/PEAKとDYNAMIC/CREST

MONITORスイッチから開くReference Monitorは、リアルタイム信号解析のための2つの精密なアナログ風計器を表示します。画面にはLyra Reference Audioの技術証明プレートを備え、クラシックHiFi機器の計器パネルのように設計されています。

4.1 HEADROOM / PEAK

瞬時ピークレベルをdBFSで測定します。スケールは-30から0 dBFS。針は高速アタック(0.60)と低速リリース(0.12)を使い、実機ピークメーターの弾道特性を再現します。デジタル表示は0 dBFSまでの残りヘッドルームを示します。無音時は左端(-30 dBFS)に戻ります。

4.2 DYNAMIC / CREST

ピークレベルとRMSレベルの差であるクレストファクターをdBで測定します。スケールは0から24 dB。値が高いほどダイナミックで圧縮の少ない音源を示し、低い値は過度な音圧処理を視覚的に示します。針は安定した低速拳動(0.08)を採用し、RMSは500 msの窓で計算されます。

4.3 測定アーキテクチャ

Android Visualizerが20,000 Hzで波形を取得します。データは[sample_0, ..., sample_N, rmsL, rmsR]としてFlutterへ送られます。Player内の単一Broadcast StreamControllerが、画面上のVUメーター、ゴニオメーター、Reference Monitorへ同時に転送し、EventChannel購読の競合を避けます。

4.4 Certificate of Direct Signal

項目	値
MODEL	LYRA HiFi Mobile Reference Player
ENGINE	Native Audio Core
DRIVER	ExoPlayer / Media3
PLATFORM	Android - Kotlin Native
SIGNAL	Direct Playback
CHANNEL	Stereo
BALANCE	Centered
DSP	None
EQ	None

5. リアルタイム表示

メインディスプレイでは3つの表示モードを利用できます。ダブルタップで切り替え、400 msのアニメーションで遷移します。

モード	操作	表示内容
回転CD	初期状態	アニメーションCD、曲情報、シークバー、再生操作
VUメーター	1回目のダブルタップ	実際のガルバノメーター物理を持つ2本の針
ゴニオメーター	2回目のダブルタップ	蛍光トレイル付きLissajous図形

5.1 VUメーター - 実アナログ計器の物理

2本の針はspring=0.08、damping=0.72、mass=1.0で動作します。支点はミリ単位で調整され、左18%、右82%、高さ83%に配置されています。

5.2 ゴニオメーター / Vectorscope

45度の正しい位相回転を持つLissajous表示です: $X=(L-R)/\sqrt{2}$ 、 $Y=(L+R)/\sqrt{2}$ 。60 fpsでフレームごとに不透明度x0.94の蛍光トレイルを残します。色は#00FF88です。

6. ユーザーインターフェース

6.1 デザイン思想

高級な物理オーディオ機器として設計されています。参照した美学はMcIntosh、Marantz、Technics。すべてのグラフィックアセットはLyra専用 Photoshopで作成されています。アプリらしく見せるのではなく、5,000ユーロ級の物理オブジェクトのように感じられることを狙っています。

6.2 音量ノブ - 直接操作と機械的ストップ

Grab offsetによりタッチ時のジャンプを防ぎ、unwrapNear()が角度の連続性を保ち、stopLockがハードリミットを制御します。進行リングは回転PNGの下にある固定レイヤーで、透明な溝から見える構造です。

6.3 シークバー

金属的なゴールドアセットで構成された専用シークバーです。ドラッグ中は位置をリアルタイムに視覚更新し、seekToは150 msでスロットルされ、指を離れた時点で最終位置を正確に送信します。ExoPlayerはonPositionChangedで即時確認するため、表示の跳ね返りを防げます。

6.4 精密ハプティクス

操作	ハプティック
Prev / Play / Next	押下時は中、リリース時は軽いフィードバック
ライブラリボタン	押下時は中、リリース時は軽いフィードバック
Powerボタン	軽いインパクト
MONITORスイッチ	中程度のインパクト
BITPERFECTボタン	状態変更時に強いインパクト
音量ノブ - 端点	最小/最大で強いインパクト
ディスプレイのダブルタップ	モード変更時に軽いインパクト

7. 試用期間とライセンス

初回起動から21日間の試用期間。期限終了後は完全ロックされます。EncryptedSharedPreferences (AES256-GCM)を用い、時計の巻き戻し対策を備えます。

コンポーネント	説明
TrialManager.kt	暗号化されたタイムスタンプと時計改ざん対策
trial_gate_screen.dart	ゴールド証明プレート付きの全画面ロック
購入シール	ワックスシール画像上のタップ領域
launchPurchaseFlow()	Google Play Billing連携

8. 技術仕様

項目	値
プラットフォーム	Android 5.0+ (API21)
フレームワーク	Flutter / Dart
ネイティブ層	Kotlin
オーディオエンジン	ExoPlayer / Media3
対応形式	MP3, FLAC, AAC, M4A, OGG, WAV, OPUS
USB DAC	USB Audio Class 1/2 (UAC1/UAC2)
音声キャプチャ	Android Visualizer - 20,000 Hz - Waveform + RMS
永続化	SharedPreferences (JSON) + EncryptedSharedPreferences (trial)
フォント	DotMatrix, DSEG7Classic
権限	READ_MEDIA_AUDIO, RECORD_AUDIO, MODIFY_AUDIO_SETTINGS
価格	10.99 EUR - 買い切り、サブスクリプションなし、広告なし

9. Flutter依存パッケージ

パッケージ	バージョン	機能
file_picker	^8.1.2	オーディオファイル選択
permission_handler	^11.3.1	実行時権限の管理
shared_preferences	^2.2.2	ライブラリと設定の保存

「アプリではなく、5,000ユーロ級の物理オーディオ機器のようなインターフェース。」
- 独立技術分析